

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME: INTERNET PROGRAMMING**

**COURSE CODE: CSA4305**

**Name: T Ganeswari**

**Reg no: 192372183**

---

### COURSE OUTCOME COVERED

**CO4:** Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

---

### QUESTIONS:

**Total Marks: 25**

#### 1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

#### 2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

#### 3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

#### 4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set,

and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

## 5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

### **Code of Conduct:**

*I T Ganeswari(192372183) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

### **Signature of the student:**

*T Ganeswari*

### **RUBRICS FOR CALCULATION:**

| Criteria                  | Weightage | Level 4 – Exemplary                                                   | Level 3 – Proficient                      | Level 2 – Developing         | Level 1 – Beginning             |
|---------------------------|-----------|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| Problem Understanding     | 20%       | Identifies all inputs, outputs, constraints accurately                | Minor missing elements                    | Partial understanding        | Incorrect interpretation        |
| Algorithm Design          | 30%       | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| Use of Control Structures | 20%       | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| Pseudocode Structure      | 10%       | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| Completeness              | 20%       | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |

# 1. XML Parsing Algorithm

Objective: Parse a hierarchical XML document, traverse its nodes, extract required fields, and display the data in a readable form while handling missing files, malformed XML, and nested elements.

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| Input           | Path of an XML file containing structured user/product records |
| Output          | Readable list/table of extracted XML values                    |
| Core Constructs | File check, parser, loops, conditions, recursive traversal     |

## Pseudocode

```
ALGORITHM Parse_XML_Document(xmlFile)
BEGIN
    IF xmlFile does not exist THEN
        DISPLAY "Error: XML file not found"
        STOP
    ENDIF

    TRY
        xmlDoc ← LOAD_AND_PARSE_XML(xmlFile)
    CATCH parsingError
        DISPLAY "Error: Invalid or malformed XML"
        STOP
    ENDTRY

    root ← GET_ROOT_ELEMENT(xmlDoc)
    DISPLAY "Root Element: " + root.name

    CALL Traverse_Node(root, 0)
END

PROCEDURE Traverse_Node(node, level)
BEGIN
    indent ← REPEAT("  ", level)

    IF node is an ELEMENT THEN
        DISPLAY indent + "<" + node.name + ">"
    ENDIF
END
```

```

        FOR EACH attribute IN node.attributes DO
            DISPLAY indent + "  @" + attribute.name + " = " +
attribute.value
        ENDFOR

        IF node contains meaningful TEXT THEN
            value ← TRIM(node.text)
            DISPLAY indent + "  Value: " + value
        ENDIF

        FOR EACH child IN node.childNodes DO
            IF child is an ELEMENT THEN
                CALL Traverse_Node(child, level + 1)
            ENDIF
        ENDFOR
    ENDIF
END

```

## Explanation

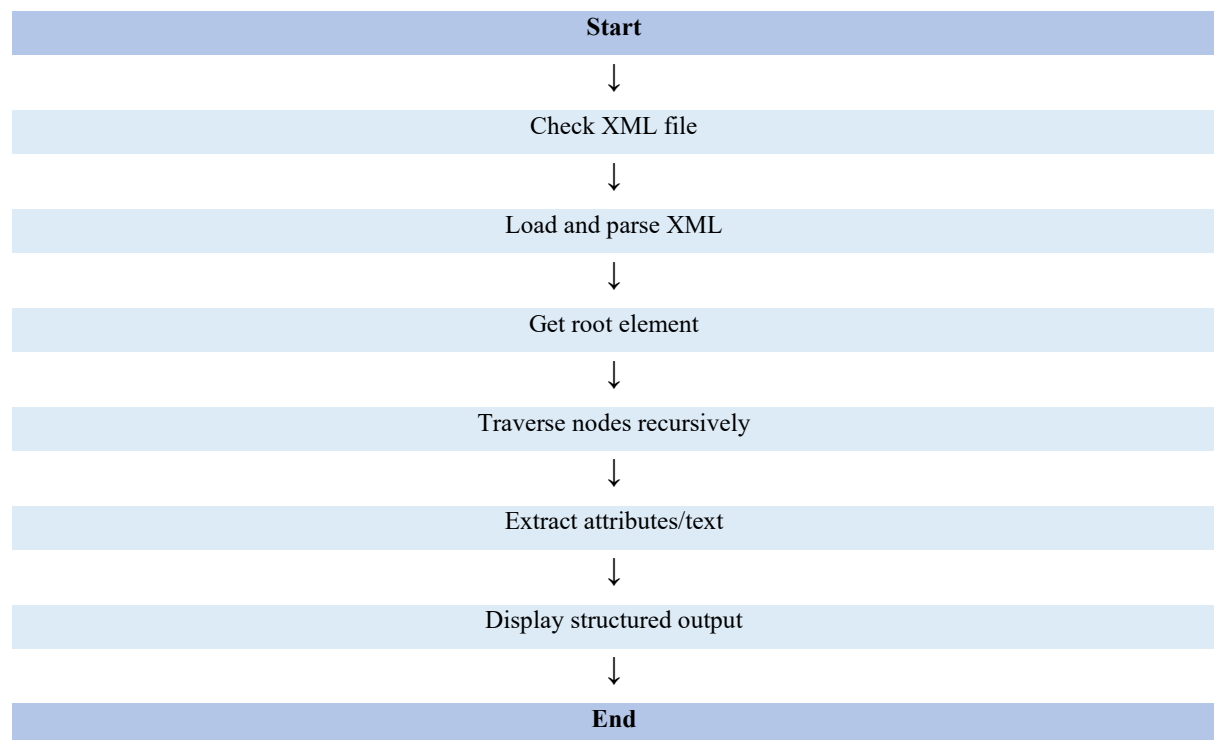
- The file is checked before parsing so that a missing input does not cause an uncontrolled failure.
- The XML parser converts the document into a tree-like structure. The root is the entry point for hierarchical traversal.
- A recursive procedure is suitable because an XML element may contain child elements at any depth.
- Attributes, text values, and child elements are handled separately, preserving the XML hierarchy.
- Malformed XML is caught and reported instead of producing incomplete output.

## Illustrative XML

```
<products>
  <product id="P101">
    <name>Laptop</name>
    <price>55000</price>
  </product>
</products>
```

Example readable output: Product ID = P101, Name = Laptop, Price = 55000.

### XML Parsing Flow



## 2. MVC Workflow

Objective: Represent the complete Model–View–Controller request lifecycle, showing separation of responsibilities between request handling, business/data logic, and presentation.

|                        |                                             |
|------------------------|---------------------------------------------|
| <b>Input</b>           | HTTP request from browser/user              |
| <b>Output</b>          | HTTP response containing rendered page/data |
| <b>Main Components</b> | Controller, Model, Database, View           |

### Pseudocode

```
ALGORITHM MVC_Request_Workflow(request)
BEGIN
    userRequest ← RECEIVE_HTTP_REQUEST(request)
    route ← IDENTIFY_ROUTE(userRequest.URL, userRequest.method)

    controller ← SELECT_CONTROLLER(route)

    IF controller is NOT FOUND THEN
        RETURN HTTP_404_RESPONSE
    ENDIF

    validatedInput ← controller.VALIDATE(userRequest.parameters)

    IF validatedInput is INVALID THEN
        viewData ← CREATE_ERROR_MESSAGES()
        response ← RENDER_VIEW("errorView", viewData)
        RETURN response
    ENDIF

    action ← controller.SELECT_ACTION(route)

    IF action requires data THEN
        model ← controller.SELECT_MODEL()
        modelResult ← model.PROCESS_BUSINESS_LOGIC(validatedInput)

        IF database access is required THEN
            dbConnection ← OPEN_DATABASE_CONNECTION()
            data ← EXECUTE_MODEL_QUERY(dbConnection, modelResult)
```

```

        CLOSE_DATABASE_CONNECTION(dbConnection)
        modelResult ← FORMAT_DATA(data)
    ENDIF
ELSE
    modelResult ← EMPTY_DATA
ENDIF

view ← SELECT_VIEW(action)
responseBody ← view.RENDER(modelResult)
response ← CREATE_HTTP_RESPONSE(responseBody, status=200)

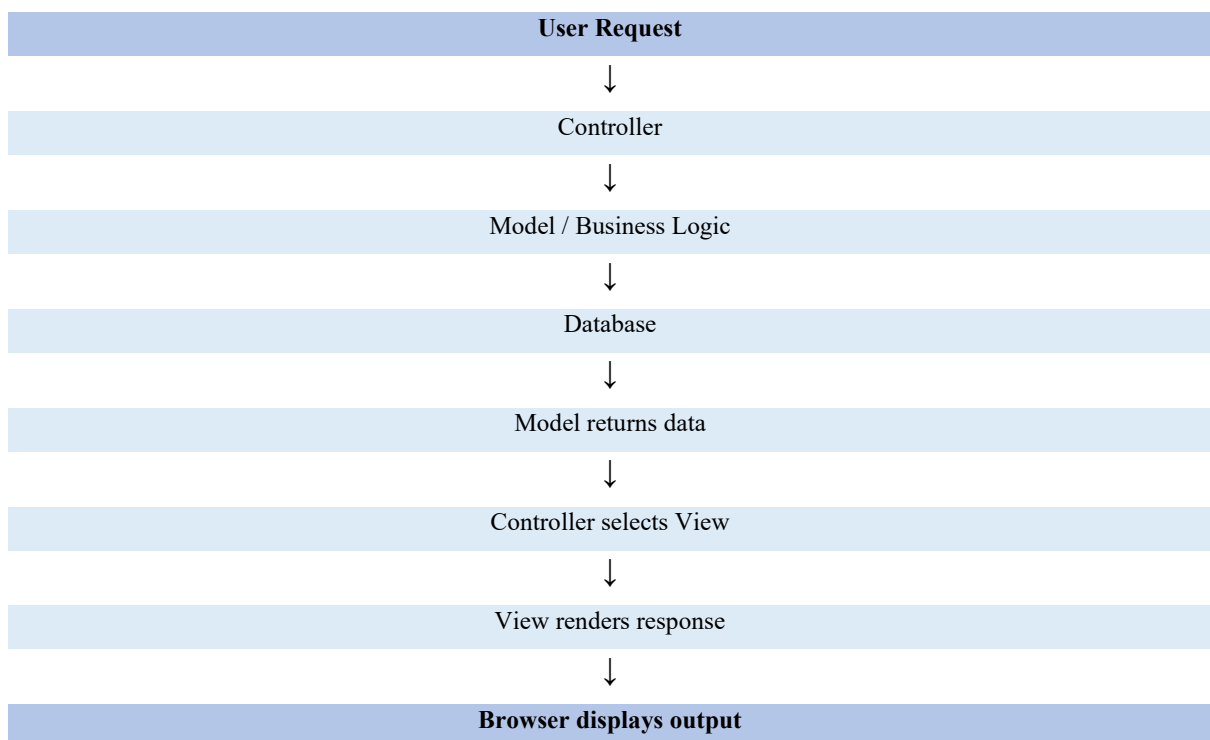
SEND response TO browser
END

```

## Component Responsibilities

| Component           | Responsibility                                                |
|---------------------|---------------------------------------------------------------|
| <b>User/Browser</b> | Sends request and receives response.                          |
| <b>Controller</b>   | Routes request, validates input, invokes Model, selects View. |
| <b>Model</b>        | Implements business logic and data access rules.              |
| <b>Database</b>     | Stores and retrieves persistent application data.             |

### MVC Request–Response Flow



### 3. JSP Request Flow

Objective: Explain how a JSP-based web application receives a browser request, executes server-side processing, optionally interacts with Servlets/JavaBeans/JDBC, and returns generated HTML.

|                |                                                              |
|----------------|--------------------------------------------------------------|
| <b>Input</b>   | Client HTTP request for a JSP/Servlet resource               |
| <b>Output</b>  | Dynamic HTML HTTP response                                   |
| <b>Backend</b> | Web container, Servlet/JSP engine, Java components, database |

### Pseudocode

```
ALGORITHM JSP_Request_Response_Flow(request)
BEGIN
    browser SENDS HTTP request TO webServer

    webServer FORWARDS request TO servletContainer

    resource ← MAP_REQUEST_URL(request.URL)

    IF resource is a Servlet THEN
        servlet ← LOAD_SERVLET(resource)

        IF servlet is not initialized THEN
            CALL servlet.init()
        ENDIF

        CALL servlet.service(request, response)
        servlet MAY:
            READ request parameters
            VALIDATE input
            CALL JavaBean/service/DAO
            ACCESS database using JDBC
            STORE result in request/session scope
            FORWARD request TO JSP
    ENDIF

    IF resource is JSP OR request is forwarded to JSP THEN
```



```
IF JSP has not been compiled OR JSP was modified THEN
    TRANSLATE JSP into Servlet source
    COMPILE generated Servlet
    LOAD and INITIALIZE generated Servlet
ENDIF

EXECUTE generated JSP Servlet
READ attributes from request/session/application
GENERATE dynamic HTML content
ENDIF

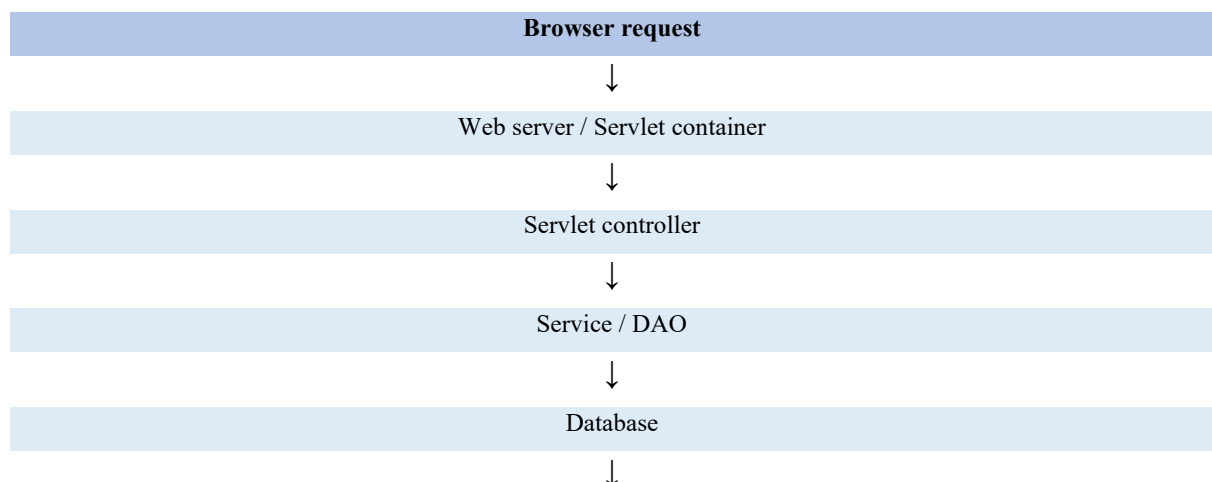
SET HTTP status and response headers
SEND generated HTML response TO browser
browser RENDERS page

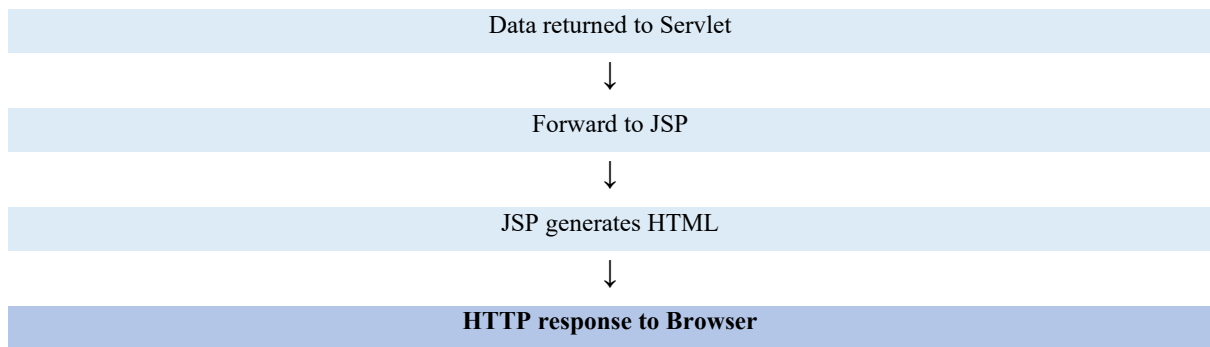
END
```

## Key Processing Notes

- A JSP is ultimately translated and compiled into a Servlet by the JSP engine; subsequent requests normally reuse the compiled servlet until the JSP changes.
- In a well-structured application, Servlets act as controllers while JSP pages focus mainly on presentation.
- Database work is preferably delegated to DAO/service classes instead of embedding JDBC code directly inside JSP.
- Request scope is useful for one request, session scope for user-specific state across requests, and application scope for shared application data.

### JSP Request Flow





## 4. Database Retrieval Algorithm

Objective: Retrieve records safely and systematically by opening a database connection, executing a query, processing the result set, displaying records, and releasing resources.

|                        |                                                                              |
|------------------------|------------------------------------------------------------------------------|
| <b>Input</b>           | Database credentials/connection string, SQL query, optional parameters       |
| <b>Output</b>          | Retrieved database records or an appropriate status/error message            |
| <b>Core Constructs</b> | Connection, prepared statement, result set loop, exception handling, cleanup |

### Pseudocode

```
ALGORITHM Retrieve_Data_From_Database(searchValue)
BEGIN
    connection ← NULL
    statement ← NULL
    resultSet ← NULL

    TRY
        driver ← LOAD_DATABASE_DRIVER()
        connection ← OPEN_CONNECTION(DB_URL, USERNAME, PASSWORD)

        IF connection failed THEN
            DISPLAY "Database connection failed"
```

```

        STOP
    ENDIF

    sql ← "SELECT id, name, email FROM users WHERE status = ?"
    statement ← PREPARE_STATEMENT(connection, sql)
    SET_PARAMETER(statement, 1, searchValue)

    resultSet ← EXECUTE_QUERY(statement)

    recordFound ← FALSE

    WHILE resultSet.HAS_NEXT() DO
        recordFound ← TRUE
        row ← resultSet.NEXT()

        id ← row.GET("id")
        name ← row.GET("name")
        email ← row.GET("email")

        DISPLAY id, name, email
    ENDWHILE

    IF recordFound = FALSE THEN
        DISPLAY "No matching records found"
    ENDIF

    CATCH databaseError
        LOG databaseError
        DISPLAY "Unable to retrieve data"
    FINALLY
        IF resultSet ≠ NULL THEN CLOSE(resultSet)
        IF statement ≠ NULL THEN CLOSE(statement)
        IF connection ≠ NULL THEN CLOSE(connection)
    ENDTRY
END

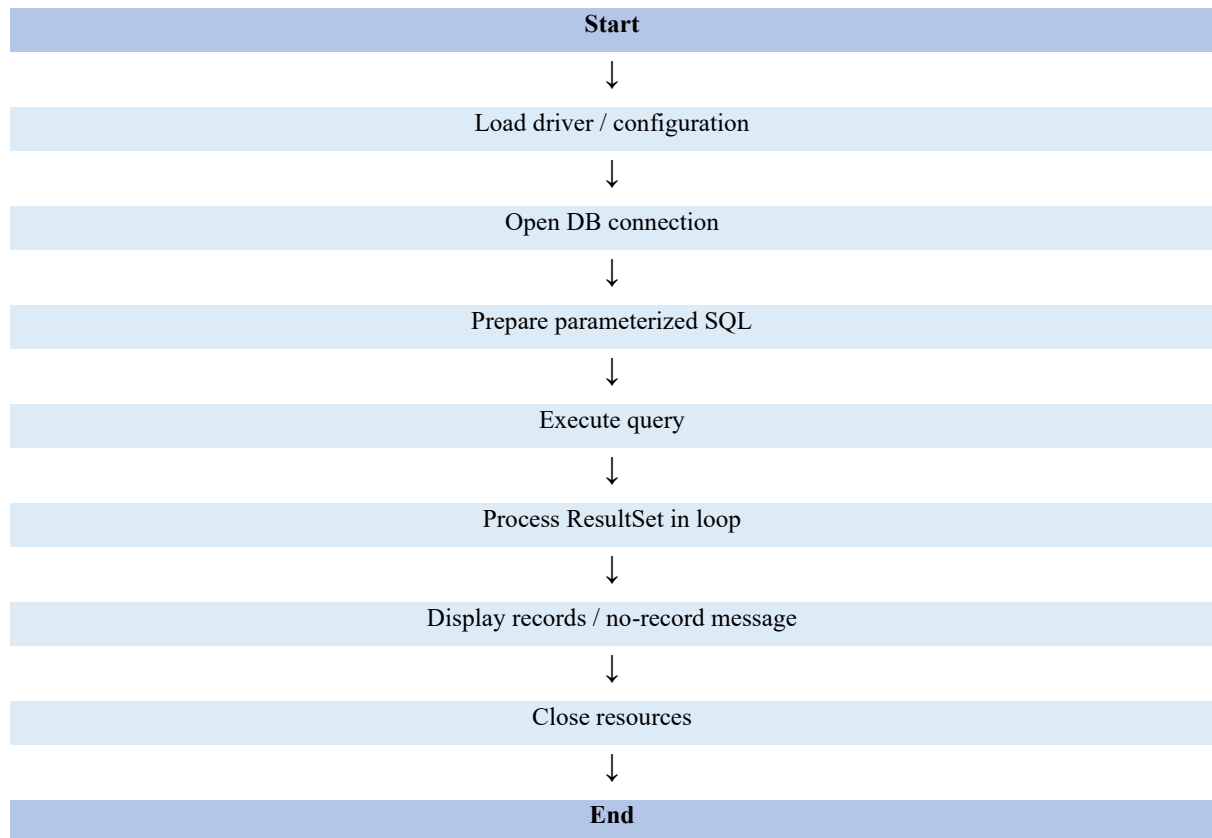
```

## Why This Design Is Strong

- A prepared statement separates SQL structure from user-supplied values and is safer than direct string concatenation.
- The WHILE loop processes any number of returned rows, including zero rows.

- TRY/CATCH handles connection or query failures, while FINALLY ensures resources are closed.
- Only required columns are selected, which can reduce unnecessary data transfer and processing.

### Database Retrieval Flow



## 5. PHP Form Validation Algorithm

Objective: Validate web-form data using PHP-style server-side checks, collect all validation errors, sanitize acceptable values, and continue processing only when every rule is satisfied.

|                   |                                                                         |
|-------------------|-------------------------------------------------------------------------|
| <b>Input</b>      | Form fields such as name, email, password and confirm password          |
| <b>Output</b>     | Validation errors or successful submission                              |
| <b>Validation</b> | Required fields, format, length/strength, matching values, sanitization |

### Pseudocode

```
ALGORITHM Validate_PHP_Form(POST)
BEGIN
    errors ← EMPTY_LIST

    IF request method ≠ "POST" THEN
        DISPLAY form
        STOP
    ENDIF

    name ← TRIM(POST["name"])
    email ← TRIM(POST["email"])
    password ← POST["password"]
    confirmPassword ← POST["confirm_password"]

    IF name is EMPTY THEN
        ADD "Name is required" TO errors
    ELSE
        name ← SANITIZE_TEXT(name)
    ENDIF

    IF email is EMPTY THEN
        ADD "Email is required" TO errors
    ELSE IF NOT VALID_EMAIL_FORMAT(email) THEN
        ADD "Enter a valid email address" TO errors
    ENDIF
```

```

IF password is EMPTY THEN
    ADD "Password is required" TO errors
ELSE
    IF LENGTH(password) < 8 THEN
        ADD "Password must contain at least 8 characters" TO errors
    ENDIF
    IF password has no uppercase letter THEN
        ADD "Password must contain an uppercase letter" TO errors
    ENDIF
    IF password has no lowercase letter THEN
        ADD "Password must contain a lowercase letter" TO errors
    ENDIF
    IF password has no digit THEN
        ADD "Password must contain a number" TO errors
    ENDIF
ENDIF

IF password ≠ confirmPassword THEN
    ADD "Passwords do not match" TO errors
ENDIF

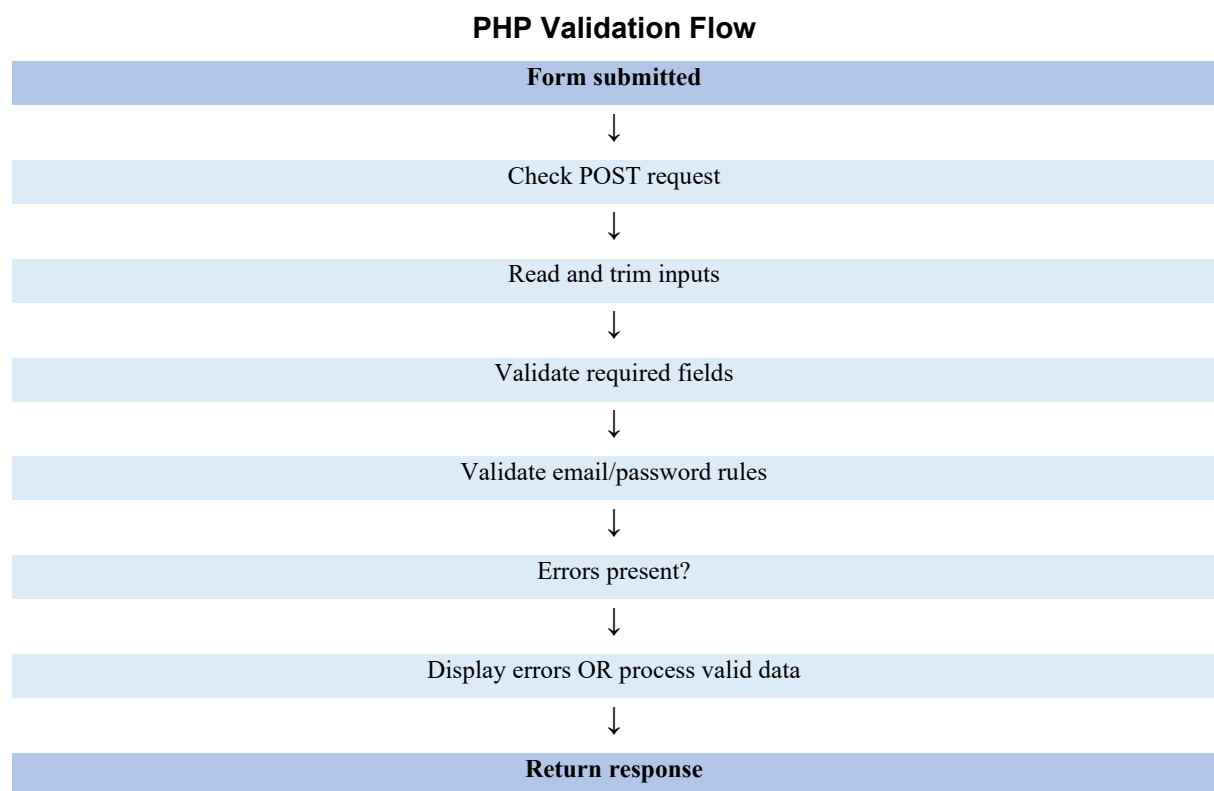
IF SIZE(errors) > 0 THEN
    FOR EACH error IN errors DO
        DISPLAY error
    ENDFOR
    DISPLAY form with previously safe values
ELSE
    passwordHash ← HASH_PASSWORD(password)
    SAVE_OR_PROCESS(name, email, passwordHash)
    DISPLAY "Form submitted successfully"
ENDIF
END

```

## Validation Logic

- Required-field validation is performed before format-specific validation.
- Email format is validated with a standard email-validation rule rather than a simple string check.
- Password checks demonstrate multiple conditions and make the control structure explicit.
- All errors are collected so the user can correct them in one attempt.

- Passwords should be hashed before storage; raw passwords should never be stored in the database.
- Client-side validation may improve usability, but server-side validation remains essential because client-side checks can be bypassed.



## Overall Conclusion

The five algorithms demonstrate systematic data-driven web processing: XML tree traversal, MVC separation of concerns, JSP/Servlet request handling, safe database retrieval, and server-side PHP validation. Each solution identifies inputs and outputs, uses suitable conditions/loops/modules, handles common error cases, and presents a clear sequence of operations.